CREDITI TIPOLOGIA D
Algoritmi di primalità: dal Miller-Rabin all'ASK

Giorgia Pazienza

A.A: 2024/2025

Indice

1 Introduzione **2**

2 Cosa sono i test di primalità **4**

3 Algoritmi che sfruttano i test di primalità **4**
3.1 Quali tipologie di algoritmi sono 4
3.2 Presupposti teorici e metodologici alla base dello sviluppo degli algoritmi . 4

4 Analisi dell'algoritmo di Miller Rabin **6**

5 Analisi dell'algoritmo AKS **10**

1 Introduzione

La matematica, in particolare la teoria dei numeri primi, riveste un ruolo fondamentale in molti ambiti della vita moderna, dalla crittografia alla sicurezza delle transazioni finanziarie.

In particolare, i numeri primi costituiscono infatti la base di molti sistemi crittografici utilizzati quotidianamente per garantire riservatezza e integrità dei dati trasmessi su internet.
Un esempio significativo è rappresentato dagli algoritmi di crittografia asimmetrica, tra cui l'RSA. Questo sistema di sicurezza sfrutta l'unicità dei numeri primi e la difficoltà di fattorizzare grandi numeri. Ogni volta che viene eseguita una transazione su un sito web, il nostro computer si affida alla crittografia RSA, che utilizza numeri primi di centinaia di cifre per cifrare e decifrare i dati in modo sicuro.
Sempre in ambito crittografico, possiamo trovare un altro esempio di utilizzo dei numeri primi nello scambio di chiavi che esegue il protocollo di Diffie Hellman, che consente a due parti, anche senza precedenti comunicazioni, di stabilire in modo sicuro una chiave segreta condivisa attraverso un canale pubblico. Tale protocollo si basa anch'esso proprietà aritmetiche dei numeri primi per garantire la robustezza della comunicazione.
Possiamo trovare un esempio di utilizzo dei numeri primi anche in un altro ambito, ovvero nel settore Finanziario. In particolare, all'interno della tecnologia delle blockchain, che basandosi su algoritmi crittografici garantisce la sicurezza nelle transizioni. Questi algoritmi crittografici a loro volta necessitano di numeri primi molto grandi per funzionare correttamente. Il test di Miller Rabin viene utilizzato per generare questi numeri primi in modo rapido e efficiente.

I numeri primi, quindi, non rappresentano solo una curiosità teorica della matematica, ma costituiscono uno strumento pratico essenziale per la protezione dei dati sensibili.

Ma che cos'è un numero primo? Fin dai primi approcci allo studio della matematica, un numero si definisce primo se non è divisibile per nessun altro numero se non per sé stesso e per 1.
Quindi possiamo capire che un numero non è primo andando a verificare che abbia dei divisori. Per farlo bisognerebbe provare con tutti i numeri $\leq \sqrt{(n)}$.
Infatti se $n = ab$ allora avremo o a $\leq \sqrt{(n)}$ o $b \leq \sqrt{(n)}$.

Esempio 1.1:, Consideriamo $n = 7$ cerchiamo due numeri interi a e b tali che $a \times b = 7$. Costatiamo che 7 può essere scritto come $7 = 1 \times 7$. Pertanto $a = 1$ e $b = 7$. Non ci sono altri numeri interi $a, b > 1$ che soddisfano $a \times b = 7$. Quindi vediamo che la seguente proprietà $a \leq \sqrt{(n)} \leq b$ non vale. Per questo affermiamo che $\boldsymbol{n = 7}$ **è un numero primo**.

Esempio 1.2: Contrariamente, per $n = 6$ cerchiamo due numeri interi a e b tali che $a \times b = 6$. Costatiamo che 6 può essere scritto come $6 = 1 \times 6; 6 = 2 \times 3$. Consideriamo $a = 2$ e $b = 3$ vediamo che $a \leq \sqrt{(n)} \leq b$ vale. Quindi affermeremo che $\boldsymbol{n = 6}$ **non è primo**.

Tutto ciò per numeri molto grandi è dispendioso, infatti se $n \approx 10^k$ allora $\sqrt{n} = \sqrt{10^k} =$

$10^{(k/2)}$. Quindi possiamo affermare che $\sqrt{n} \approx 10^{(k/2)}$. Da qui nascono problemi come la primalità e la fattorizzazione, due questioni cruciali nella crittografia moderna.

2 Cosa sono i test di primalità

I test di primalità sono algoritmi che permettono di verificare se un numero intero è primo oppure no. Questi test sono alla base di molti algoritmi crittografici, poiché identificare numeri primi è essenziale per la generazione di chiavi crittografiche sicure.

3 Algoritmi che sfruttano i test di primalità

3.1 Quali tipologie di algoritmi sono

Gli algoritmi di primalità si dividono in due categorie principali: deterministici e probabilistici.

- **Algoritmi deterministici:** Questi algoritmi, dato un input, producono sempre lo stesso risultato e non contengono elementi di casualità.
  Ad esempio, applicando un algoritmo deterministico a un numero k volte, l'output sarà sempre lo stesso, indicando con certezza se n è primo o no.

- **Algoritmi probabilistici:** Questi algoritmi, invece, includono elementi di casualità ed, applicando il test più volte, è possibile ottenere risultati diversi. Non forniscono una risposta certa, ma una probabilità che il numero sia primo. Tuttavia, ripetendo il test più volte, la probabilità di errore diminuisce in modo esponenziale.

Fin dal Seicento, sono stati ideati vari metodi per testare la primalità dei numeri. Alcuni esempi classici sono il test di Fermat e il test di Wilson. Noi ci focalizzeremo sul Test di Fermat.

3.2 Presupposti teorici e metodologici alla base dello sviluppo degli algoritmi

Il piccolo teorema di Fermat è uno dei punti di partenza per molti algoritmi di primalità.

Esso afferma che,

$$\textit{dato un numero primo } p \textit{ ed un numero } a \in \mathbb{Z}.$$
$$\textit{Se } p \nmid a \textit{ allora si calcola } a^{(p-1)} \equiv 1 \textit{ mod } p.$$

Questo teorema è utile per dimostrare che un numero è probabilmente primo.

Esempio 3.1: Dato un numero primo p = 11 e un numero intero a = 2. Per il teorema di fermat avrò

$$2^{10} \textit{ mod } 11 \equiv 1 \textit{ mod } 11$$

Segue un codice, implementato a partire dal Piccolo Teorema di Fermat spiegato precedentemente. Il seguente codice calcolerà il risultato numerico per dati input specifici.

Algorithm 1 Implementazione in Python del piccolo teorema di Fermat

```
1: // Importiamo una libreria che permette di generare numeri casuali
2: import randoms
3: function PICCOLO_TEOREMA_DI_FERMAT(n)
4:     b = random.randint(2, n − 1)
5:     // La funzione integrata pow() calcola l'elevamento a potenza modulare
6:     if pow(b, n − 1, n) != 1 then
7:         return False
8:     else
9:         return True
10:    end if
11: end function
```

Sia n un intero positivo. La funzione restituisce un valore booleano che classifica n come segue:

$$\text{output} = \begin{cases} \text{True} & \text{se } n \text{ è probabilmente primo,} \\ \text{False} & \text{se } n \text{ è certamente composto.} \end{cases}$$

Ad esempio, per $n = 11$ la funzione restituisce `True`, mentre per $n = 16$ restituisce `False`.

N.B: un numero è detto probabilmente primo quando tutte le verifiche probabilistiche effettuate decretano sia primo, pur non potendo escludere completamente che sia composto.

4 Analisi dell'algoritmo di Miller Rabin

L'algoritmo di Miller-Rabin rappresenta un raffinamento del piccolo teorema di Fermat per la verifica della primalità dei numeri interi.
A differenza del test di Fermat, il quale fornisce una condizione necessaria ma non sufficiente per la primalità, Miller-Rabin introduce un test probabilistico che riduce significativamente il rischio di falsi positivi (i cosiddetti "numeri di Carmichael" citati precedentemente). Esistono infiniti interi composti che si fingono primi rispetto al test di Fermat. Questi sono chiamati "a-pseudoprimi".

Il teorema afferma:

Sia n un numero primo dispari. Posso scrivere un numero pari nel seguente modo
$$n - 1 = 2^s \times r$$

dove:

- s è un numero intero qualsiasi, opportuno
- r è un numero intero dispari

Esempio 4.1: Per $n = 5$ avremo $n - 1 = 4$ che possiamo riscrivere come $2^2 \times 1$. Quindi scriveremo che:

$$\begin{gathered} n - 1 = 2^s \times r \\ \downarrow \\ 4 = 2^2 \times 1 \end{gathered}$$

Sia a un coprimo di n, ciò significa che il loro *MCD(a, n) = 1* (ovvero non hanno divisori in comune) e sia $b = a^m$

Avendo $a < n$ un numero qualunque e sia $b = a^m$ ed elevo b alle varie potenze di 2, ovvero:

$$b = b^2;\ b^{2^2};\ b^{2^3};\ ...;\ b^{2^k};\ \exists k$$

Per quanto detto precedentemente avrò

$$\begin{aligned} b^{2^k} &= (a^m)^{2^k} \\ b^{2^k} &= a^{2^k m} \\ b^{2^k} &= a^{n-1} \end{aligned}$$

Per il piccolo teorema di fermat noi affermeremo che

$$a^{n-1} \equiv 1 \ mod\ n \text{ se } n \text{ è primo}$$

Esempio 4.2: Applichiamo il test di Miller–Rabin al numero

$$n = 13.$$

Calcolo $n - 1 = 12$ e lo decompongo nel seguente modo:

$$n - 1 = 12 = 2^2 \cdot 3.$$

dove:

- s = 2,
- m = 3.

Scegliamo una base valida nell'intervallo $[2, n-2]$, o anche detto $2 \leq a \leq n-2$

Prendiamo

$$a = 2.$$

Definiamo

$$b = a^m \bmod n = 2^3 \bmod 13 = 8 \bmod 13 = 8$$

Quindi:

$$b = 8.$$

Calcolo tutti i quadrati successivi:
Dobbiamo considerare i valori b^{2^k} per $k = 0, 1, \ldots, s-1$ (qui $s - 1 = 1$).

$$b^{2^0} = b = 8,$$

$$b^{2^1} = b^2 = 8^2 = 64 \equiv 64 - 5 \cdot 13 = 64 - 65 = -1 \equiv 12 \pmod{13}.$$

$$b^{2^2} = (b^2)^2 = 12^2 = 144 \equiv 1 \pmod{13},$$

il che è coerente con il fatto che $a^{n-1} = a^{12} \equiv 1 \pmod{13}$ quando n è primo.

Tabella di osservazione

k	**Espressione**	**Risultato mod** 13
0	$b^{2^0} = a^m$	8
1	$b^{2^1} = a^{2m}$	12
2	$b^{2^2} = a^{4m}$	1

Per controllare che il risultato ottenuto sia corretto, calcoliamo direttamente

$$a^{n-1} = 2^{12} \pmod{13}.$$

Questo serve per verificare che il numero $n = 13$ rispetti il *piccolo teorema di Fermat*, secondo cui, se n è primo, allora per ogni a coprimo con n vale:

$$a^{n-1} \equiv 1 \pmod{n}.$$

Calcoliamo ora passo per passo.

$$2^6 = 64 \equiv 12 \pmod{13}.$$

Abbiamo scelto di calcolare prima 2^6 perché 2^{12} è un numero grande e può essere riscritto come

$$2^{12} = (2^6)^2.$$

In questo modo possiamo sfruttare il risultato precedente e ridurre il calcolo.

$$2^{12} = (2^6)^2 \equiv 12^2 = 144 \equiv 1 \pmod{13}.$$

Il risultato $2^{12} \equiv 1 \pmod{13}$ conferma che 13 si comporta come un numero primo rispetto alla base $a = 2$. Questo è coerente con il piccolo teorema di Fermat: infatti, se n è primo, ogni base a coprima con n deve soddisfare $a^{n-1} \equiv 1 \pmod{n}$.

Conclusioni: Nel test di Miller–Rabin abbiamo visto che, scegliendo $a = 2$, si ottiene

$$b^{2^1} \equiv 12 \equiv n - 1 \pmod{13}.$$

[Questo significa che nella catena dei quadrati compare proprio $n - 1$], quindi il numero $n = 13$ supera il test: è *probabilmente primo*. Poiché 13 è davvero un numero primo, il test ha dato un risultato corretto.

Nota. Per aumentare la sicurezza del test si ripete la procedura su più basi distinte a.

Il seguente codice calcolerà il risultato numerico per dati input specifici.

Algorithm 2 Test di Miller-Rabin

```
    from random import randrange
 2:
    function Miller_Rabin(n : int)
 4:     if n ∈ [2, 3, 5, 7] then
            return True
 6:     else if n < 10 then
            return False
 8:     end if
        s ← 0
10:     m ← n − 1
        while m mod 2 == 0 do
12:         m ← m ≫ 1
            s ← s + 1
14:     end while
        assert (2^s · m == n − 1)
16:
        function TEST_ROBUSTEZZA(a)
18:         if (a^m) mod n == 1 then
                return False
20:         end if
            for i ∈ [0, s − 1] do
22:             if (a^(2^i · m)) mod n == n − 1 then
                    return False
24:             end if
            end for
26:         return True
        end function
28:     for i ∈ range(8) do
            a ← randrange(2, n)
30:         if prova_composità(a) then
                return False
32:         end if
        end for
34:     return True
    end function
```

5 Analisi dell'algoritmo AKS

Il test di Primalità AKS, sviluppato nel 2002 in un articolo chiamato "Prime is in P", prende il nome dai suoi creatori Agrawal-Kayal-Saxena, tre informatici dell'Indian Istitute of Tecnology Kanpur.
Quest'algoritmo è capace di verificare se un numero è primo con caratteristiche che lo rendono unico sotto diversi punti di vista.
L'algoritmo ha 4 proprietà chiave:

- è *poliomiale*, cioè il suo tempo di esecuzione cresce rispetto al numero di cifre dell'input. Ciò significa che all'aumentare delle dimensioni del numero da analizzare, il tempo richiesto non esplode in maniera esponenziale, ma rimane gestibile da un punto di vista computazionale.
- è *deterministico*, ovvero non si basa su un caso probabilistico. Ciò significa che restituisce in output la risposta primo se effettivamente il numero analizzato è primo.
- è *generale*, ovvero funziona per tutti i numeri naturali. Non ha "limitazioni" particolari di classi che deve rispettare.
- è *non condizionato*, ovvero la sua efficacia non dipende da formule matematiche ancora da dimostrare.

Diamo quindi un enunciato del Teorema:

Siano $a \in \mathbb{Z}$, $n \in \mathbb{N}$, $n \leq 2$ coprimi. Allora n è primo se e solo se in $\mathbb{Z}/_{(n)}[X]$

$$(X + a)^n = X^n + a$$

Quindi, per testare se n è primo, si sceglie a coprimo con n e si testa l'equazione

$$(X + a)^n = X^n + a.$$

Questa equivalenza è basata sul teorema binomiale. Infatti andiamo ad espandere il binomio $(X + a)^n$ tramite la formula del binomio di newton

$$(X + a)^n = \sum_{k=0}^{n} \binom{n}{k} X^{n-k} a^k.$$

L'espansione completa di $(X + a)^n$ genera un polinomio con $n + 1$ coefficienti, quindi la verifica diretta dell'identità richiede tempo $O(n)$. Poiché la dimensione dell'input è $\log n$ (il numero di bit necessari per rappresentare n), una complessità lineare in n è esponenziale rispetto alla dimensione dell'input e quindi impraticabile. Gli algoritmi di primalità efficienti, come AKS, devono invece avere complessità polinomiale in $\log n$, da cui la necessità di ridurre il grado dei polinomi tramite la riduzione modulo $X^r - 1$.

Esempio 5.1: Applichiamo il Test AKS al numero

$$n = 13.$$

Scegliamo un intero a coprimo con n, ad esempio $a = 2$. Secondo il teorema enunciato:

$$n \text{ è primo} \quad \iff \quad (X + a)^n \equiv X^n + a \text{ in } \mathbb{Z}/(n)[X].$$

Verifichiamo dei coefficienti binomiali modulo n Sviluppiamo il binomio di Newton:

$$(X + 2)^{13} = \sum_{k=0}^{13} \binom{13}{k} X^{13-k} \cdot 2^k = X^{13} + \sum_{k=1}^{12} \binom{13}{k} X^{13-k} \cdot 2^k + 2^{13}.$$

Per $1 \leq k \leq 12$, i coefficienti binomiali $\binom{13}{k}$ sono tutti multipli di 13. Infatti, $\binom{13}{k} = \frac{13 \cdot 12 \cdot \cdots \cdot (13-k+1)}{k}$ contiene 13 al numeratore e $k < 13$, quindi

$$\binom{13}{k} \equiv 0 \pmod{13}.$$

Applichiamo del piccolo teorema di Fermat Poiché $MCD(2, 13) = 1$, il piccolo teorema di Fermat garantisce che

$$2^{13} \equiv 2 \pmod{13}.$$

Sostituendo i risultati nei termini del binomio otteniamo:

$$(X + 2)^{13} \equiv X^{13} + 2 \pmod{13}.$$

Poiché la congruenza $(X + a)^{13} \equiv X^{13} + a$ vale in $\mathbb{Z}/(13)[X]$ per $a = 2$ coprimo con 13, il teorema implica che

$$n = 13 \text{ è primo.}$$

Dopo aver enunciato il teorema, è possibile presentare l'algoritmo che lo implementa.

Algorithm 3 Test AKS

```
// Importiamo le librerie necessarie
import math
from sympy import totient, symbols, Poly
X = symbols("X")

function IS_PERFECT_POWER(n)
    for b = 2 to ⌊log_2(n)⌋ + 1 do
        a = ⌊n^(1/b)⌋
        if a^b = n then
            return True
        end if
    end for
    return False
end function

function FIND_R(n)
    max_k = ⌊(log n)^2⌋
    for r = 2 to n − 1 do
        ok = True
        for k = 1 to max_k do
            if pow(n, k, r) = 1 then
                ok = False
                break
            end if
        end for
        if ok = True then
            return r
        end if
    end for
    return None
end function
```

```
1: function AKS(n)
2:     if n ≤ 2 then
3:         return "n deve essere > 2"
4:     end if
5:     if IS_PERFECT_POWER(n) then
6:         return "COMPOSTO"
7:     end if
8:     r = FIND_R(n)
9:     for a = 2 to r do
10:        g = gcd(a, n)
11:        if 1 < g < n then
12:            return "COMPOSTO"
13:        end if
14:    end for
15:    if n ≤ r then
16:        return "PRIMO"
17:    end if
18:    limit = ⌊φ(r) · log n⌋
19:    for a = 1 to limit do
20:        left = (X + a)^n mod (X^r − 1) mod n
21:        right = (X^n + a) mod (X^r − 1) mod n
22:        if left ≠ right then
23:            return "COMPOSTO"
24:        end if
25:    end for
26:    return "PRIMO"
27: end function
```